Ders 3 Genişletilmiş Notu: Dönem Projesi, CASE Araçları, CMMI Düzeyleri ve Kullanım Senaryolarına Giriş

Ders 3 Genişletilmiş Notu: Dönem Projesi, CASE Araçları, CMMI Düzeyleri ve Kullanım Senaryolarına Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Şimdiye Kadar Neler Gördük?
- 2. Dönem Projesi: Amaç ve İstenen Çalışmalar
- 3. Proje Konuları
- 4. Modül Kavramı ve Uygulamanın Kapsamı
- 5. Analiz ve Tasarım Neden Önemli? Salıncak Hikâyesi
- 6. Sistem Geliştirme Yaşam Döngüsü
- 7. Temel ve Destek (Şemsiye) Aktiviteleri, Ölçme
- 8. CASE Araçları
- 9. Süreç Sertifikasyonu: CMM ve CMMI
- 10. CMMI Düzeyleri
- 11. Türkiye'deki Durum
- 12. Nesneye Yönelik Gereksinim Analizine Giriş: Kullanıcı Hikâyeleri ve Use Case
- 13. Fonksiyonel ve Fonksiyonel Olmayan Gereksinimler
- 14. Use Case Belgesinin Yapısı

1. Şimdiye Kadar Neler Gördük?

Hoca derse, önceki iki derste işlenen konuları hatırlatarak başladı. İlk hafta ders içeriği gözden geçirilmiş, ardından yazılım mühendisliği kavramları, yazılım geliştirme süreç modelleri ve çevik (agile) yöntemler işlenmişti. Sırada gereksinim mühendisliği (requirements engineering) ve tasarım konuları vardı.

Hoca, dersin biraz hızlı ilerlediğini, bu hafta aradaki bazı eksikleri tamamlayacağını ve gereksinim mühendisliğine bir sonraki hafta geçmeden önce biraz modellemeye gireceğini söyledi. Bu derste senaryo, use case diyagramları, sınıf şeması ve UML kavramları, gereksinim analizinin tanımlanmasıyla birlikte diğer derslere göre daha hızlı verilecekti. UML durum şeması ve etkinlik şeması ise araştırma görevlilerinin yürüteceği uygulama derslerinde, baştan sona bir örnek üzerinde çalışılacaktı.

2. Dönem Projesi: Amaç ve İstenen Çalışmalar

2.1 Sistem Analizi dersindeki projeden farkı

Yazılım Mühendisliği dersinde her yıl bir dönem projesi verilir, ama bu proje Sistem Analizi dersindekinden farklıdır. Sistem Analizi dersinde gruplar konuyu kendileri öneriyor ve klasik (yapısal) yöntemle ya da nesneye yönelik yöntemle çalışmakta serbest bırakılıyordu. Bu derste ise konuları hocalar veriyor ve üç farklı konu var.

Asıl fark yöntemdedir. Bu projede analiz, modelleme, tasarım ve program kodlaması baştan sona nesneye yönelik (object-oriented) yapılır. Tasarım prensiplerinde de hep nesneye yönelik yaklaşım kullanılır.

> **Hocanın vurgusu:** Projenin temel amacı, nesneye dayalı yönteme göre analizin, modellemenin ve tasarımın yapılmasıdır. Tasarımdan sonra seçilen en az bir modül istenen dilde ve platformda kodlanır ve test edilir. Sistemin tamamı çalışmak zorunda değildir, ama bir bölümü çalışır hale getirilir.

2.2 Projede istenenler

Hoca, projede görmek istediklerini proje tanımı belgesi üzerinden sıraladı:

- **Proje planı:** Yazılım Mühendisliği dersinde görülecek proje planı öğeleri. Projenin kabul kısıtları, proje alanı (kapsamı), iş zaman çizelgesi (Sistem Analizi dersinde görülen Gantt diyagramı), ekip organizasyon tablosu ve risk tablosu.
- **İsteklerin modellenmesi:** Gereksinim analizi sonucunda bir kullanım senaryosu (use case) oluşturulur. Hem kullanım senaryosu belgesi hem de use case diyagramı istenir.
- **Sınıf diyagramı (class diagram):** UML çizimleri kapsamında, Nesneye Yönelik Analiz dersinde görülen sınıf diyagramı. O dersi almamış olanlar eksiklerini tamamlayacak, bu derste de genel bilgi verilecek.
- **Nesneye dayalı tasarım:** Senaryolar sınıf diyagramı, sıralama diyagramı (sequence diagram), etkinlik diyagramı (activity diagram) ve durum diyagramı (state diagram) ile zenginleştirilir. Sıralama diyagramı diğer derslerden bilinir, etkinlik ve durum diyagramları büyük ölçüde bu derste görülecektir.
- **Tasarım kalıbı (design pattern):** Kullanıldıysa hangisinin kullanıldığı belirtilir.
- **Kodlama:** En az bir modül, nesneye dayalı bir dilde kodlanır.
- **Birim testi (unit test):** Kodlanan modül için iki birim testi tasarlanır. Testler kaynak kodlara ve rapora eklenir, raporda bu testlerle ilgili bilgi verilir.

2.3 Varsayımlar, kabuller ve kısıtlar

Konu tanımları bilerek geniş tutulmuştur. Hoca, küçük ayrıntıları tek tek sormak yerine grupların varsayım, kabul ve kısıt (assumptions and constraints) ekleyebileceğini söyledi. Bir konuyu birkaç grup yapacağı için her grubun küçük kabulleri, altyapıda kullandığı araçlar farklı olabilir. Bu yüzden her projenin başında "varsayımlar şunlar, kabuller şunlar, kısıtlar şunlar" diye bir bölüm olması beklenir.

> **Hocanın vurgusu:** Küçük varsayımlar yapılabilir, ama projeyi tamamen küçültecek ya da sistemin şeklini çok değiştirecek bir şeyi "ben böyle varsaydım" diye yazmayın. Öte yandan çok büyük ölçekli yeni özellikler de eklemeyin.

2.4 Diyagramlar bir CASE aracıyla çizilir

Hoca, önceki derslerde bahsettiği CASE (Computer Aided Software Engineering, bilgisayar destekli yazılım mühendisliği) kavramını hatırlattı. Projedeki UML şemaları elle değil, basit bir CASE aracıyla çizilecek. Önerilen araç açık kaynaklı Violet UML'dir, ama başka bir araç da kullanılabilir.

Bir araç kullanılmasının iki gerekçesi var:

- **Standart çizim:** Diyagramların elle, standart dışı çizilmesini önlemek.
- **Diyagramlar arası tutarlılık:** Analizden tasarıma geçerken use case diyagramı ile sınıf diyagramı arasında, özellikle de sınıf diyagramı ile sıralama diyagramı arasında doğru bir geçişi ve kontrolü sağlamak.

> **Hocanın vurgusu:** Violet önerilir, başka bir şeyle de hazırlanabilir, ama mutlaka bir araç kullanılmalıdır.

2.5 Uygulama dersleri

Soru: Derste söylenen iki uygulama dersinin içeriğinde ne olacak? **Cevap:** Derslerde şemalar, diyagramlar, çizim kuralları tek tek, bağımsız örneklerle görülüyor. Uygulama derslerinde ise

araştırma görevlileri, gereksinim analizi tarafındaki diyagramlarla ve durum ile etkinlik diyagramlarıyla baştan sona bir problemi çözecek. Amaç, projelerde analiz ve tasarım sırasındaki modellemelere örnek olacak problem çözümleri göstermektir. Bu uygulamalar özellikle Nesneye Yönelik Programlama dersini almamış öğrenciler için önemlidir, çünkü nesneye yönelik yaklaşımla problem çözmeyi ve bunu diyagramlara aktarmayı gösterir.

Hoca, uygulama dersinin yazılım ölçütleri konusuyla yer değiştirmesini düşündüğünü de belirtti. Gerekçesi akademikti: Yazılım ölçütleri diyagram çizimiyle ilgili olmayan bağımsız bir konudur, uygulama dersindeki içerik ise projeler için daha erken gereklidir.

3. Proje Konuları

Hoca üç konuyu proje tanımı belgesinden özetledi. Konu metinlerinin tamamını okumadı, belgeyi paylaşacağını söyledi.

3.1 Seyahat acentesi bilet ve tatil sistemi

- Havayolu, demiryolu ve tur biletleri satılıyor.
- İleride başka firmaların biletleri de satılabilecek. Hoca burada bilerek açık bir nokta bıraktıklarını söyledi.
- Bilet almak isteyen müşteri karşısında sistemin nasıl davrandığı belgede anlatılıyor.
- Bir puan sistemi var: Tatile gittikçe ya da bilet aldıkça puan toplanıyor. Biriken puanlar ödemede kullanılabiliyor. Bu puan sisteminin projeye eklenmesi isteniyor.
- Ödeme türleri: nakit, kredi kartı, çek.

3.2 Yazılım evi insan kaynakları sistemi

- Bir yazılım evinin insan kaynakları yönetimi için çalışanlar hakkında bilgi tutuluyor. Çalışanlar sisteme kaydediliyor.
- Şirkette başlatma ve bitirme kararları yönetim tarafından verilen projeler yürütülüyor. Bu projelerin insan kaynağı takibi yapılıyor.
- Görev farklılıkları var: analist, tasarımcı, programcı, yönetici. Bu görev bilgileri de takip ediliyor.
- Çalışanın hangi projeye atanacağı belirleniyor. Proje bittiğinde çalışan başka bir projeye atanabilir, ama herkesin yeni bir projeye atanması zorunlu değil, işten çıkarılanlar da olabilir.
- Maaş ve tazminat hesapları bu projenin dışındadır. Bu ayrıntılı mali hesaplar muhasebe programlarına, yani mali uygulamayı yazan gruba bırakılmıştır. Proje yalnızca çalışanın hangi grupta (çalışan türünde) olduğunu atar.

Hoca bu konuyu, temelde bir yazılım şirketinin insan kaynaklarıyla ve çalışanların projelere atanmasıyla ilgili bir uygulama olarak özetledi.

3.3 Medya takip sistemi

Medya takip şirketleri medyadaki haberleri izleyip müşterileriyle ilgili haberleri onlara rapor eder. Projede böyle bir şirket için yazılım geliştirildiği düşünülür.

- Şirket çeşitli tipte medya organlarını takip ediyor: radyolar, televizyonlar, gazeteler, dergiler. Günümüze uygun olarak internet siteleri ve blog sayfaları da eklenebilir. Her tipte değişik sayıda yayın takip ediliyor.
- Haber takibi isteyen müşteriler şirket ya da şahıs olabilir. Başka müşteri tipi yok.
- Standart olarak günlük, haftalık ve aylık raporlar hazırlanıyor. Standart raporların fiyatları belli.
- Müşteriye özel raporlarda periyot farklı belirlenebilir. Periyot ve fiyat anlaşma imzalanırken belirlenir ve sonradan değişmez.
- Takip edilen medya organları bir veri tabanında tutulur. Yeni çıkanlar eklenir, değişenler güncellenir, kapananlar silinir.
- Şirket, müşterilerin itibarı ve bilinirliği gibi konularda anketler düzenleyip sonuçlarını müşterilere bildirir. Anketler ayrıca ücretlendirilir. Kayıtlı anketörlerin iş planı sistem üzerinden yapılır.

- Medya organlarını takip için ortak bir arayüz kullanılır. Bir müşteri için rapor hazırlanacağında sistem haberleri bu arayüzle sorgular. Sonuç bir belge halinde müşteriye gönderilir.

4. Modül Kavramı ve Uygulamanın Kapsamı

Öğrenci sorularıyla, projede "modül" kavramının ne anlama geldiği netleşti.

Soru: Proje bir uygulama olacak, yani düğmeleri olan çalışan bir program mı? **Cevap:** Evet, bir uygulama olacak. Ama konular büyük projelerdir, bir modülünün kodlanması yeterlidir. Programa yukarıdan bir giriş olur ve girişte tüm modüllere yönlendirme yapılabilir. Ama aşağıda yalnızca seçilen modül çalışır. Örneğin seyahat acentesinde "şu modülü seçtik, bu modülü kodladık" denir ve dönem sonunda bu modülün çalıştırılması gösterilir.

Soru: Seyahat acentesi konusunda modüller, belgede sayılan maddeler mi? **Cevap:** Hayır. Belgedeki otobüs, havayolu, tur biletleri gibi maddelerin her biri bir modül değildir. Bunlar hocaların açıklama yapmak zorunda hissettiği noktalar, yani ipuçlarıdır. Modülleri gruplar belirler. Seyahat acentesi için ödeme modülü, rezervasyon modülü ya da puanlama sistemi birer modül olabilir. Puanlama seçilirse, biriken puanların sonraki bilet alımında kullandırılması da işin içindedir.

Soru: Modül için başka bir örnek verilebilir mi? **Cevap:** Medya takip sisteminde temel alt işlemler şöyle oluşturulabilir: hangi müşterilerin takip edileceği ve müşterilerle yapılan anlaşmalar, takip edilecek medya organlarının seçilmesi ve belirlenmesi, bu iki tarafın eşleştirilmesi.

Hoca bunun bütün konular için geçerli olduğunu vurguladı. Verilen metinler varsayımlar ve özelliklerin açıklamasıdır. Alt modüller verilmez. Bir uygulamayı gördüğünüzde altında hangi modüllerin olduğunu sizin çıkarmanız beklenir. Daha kalabalık bir ekipte kodlanacak modül daha kapsamlı tek bir modül ya da iki küçük modül olabilir.

5. Analiz ve Tasarım Neden Önemli? Salıncak Hikâyesi

Proje tanıtımından sonra hoca analize giriş yaptı. Bir bilgi sisteminin, özellikle bir yazılım sisteminin iyi oluşturulması için önce ne istendiğinin anlaşılması gerekir. Hoca, gereksinim analizine geçmeden önce analiz ve tasarımın neden başta önemli olduğunu, neden önce modelleme yapılıp sonra program yazımına geçildiğini, Sistem Analizi ve Tasarımı dersinde de gösterdiği klasik bir örnekle anlattı: salıncak hikâyesi.

Hikâyede müşterinin istediği aslında çok basittir: bir ağaca asılmış basit bir salıncak, doğru çalışır halde. Ama müşteri yaptığı işi kendi dünyası etrafında, bütün ayrıntılarıyla ve çok karmaşık anlatır. Bunun sonucunda bir hata zinciri oluşur:

1. Proje lideri müşteriden işin nasıl yapılacağını anlamalı, doğru planlamalarla doğru anlatmalıdır. Karmaşık anlatım karşısında yanlış algılar.
2. Analist, gereksinim analizinde müşterinin çok ayrıntılı anlattıklarını daha da karmaşık hale getirir.
3. Tasarım tamamen hatalı oluşturulabilir.
4. Yapılanlar programcıya da hatalı anlatılırsa programcı bambaşka bir sonuç çıkarır.
5. Ortaya istenmeyen bir ürün çıkar.

Hoca hikâyeye bir boyut daha ekledi: Bütün bunlar yapılırken belgeleme, yani proje dokümantasyonu yapılmamışsa ne olacak? Bir sınıf diyagramı, ER diyagramı, varlıklar arasındaki ilişkiler, veri tabanı yapısı gibi belgeler yoksa, sonradan sistem üzerinde çalışacak kişiler ona hiçbir şekilde müdahale edemez ya da çok zorlanır. Bir takım yazılım yeniden mühendisliği (software re-engineering) işlemleri yapmak zorunda kalırlar.

Hoca bu hikâyeyi, geçen hafta çevik ilkeler arasında görülen basitlik (simplicity) ilkesine bağladı: Ayrıntılara boğulmadan önce ne istendiği ve ne yapılacağı düşünülmelidir. Müşteri ilk etapta bütün ayrıntıları anlatmamalıdır.

> **Hocanın vurgusu:** Müşteri istediklerini basitten karmaşığa doğru anlatmalı, karmaşık süreçleri gerekli yerlerde, soruldukça açmalıdır. Analizi yapan kişilerin de

iletişim ve soru sorma konusunda başarılı olması gerekir. Bu kavramlar yanlış anlaşıldıkça doğru çalışan bir sistem elde edilemez.

6. Sistem Geliştirme Yaşam Döngüsü

Gereksinim analizinde ilk iş sistemin yapısını anlamaktır. Hoca geçen hafta söylediğini tekrarladı: Yazılım, bir kurum içinde yaşayan bir sistem olarak görülmelidir. Önce kurumun yapısı, sonra kurumda nasıl bir çalışma sistemi olduğu görülür. Ardından bu sistem içinde hangi tekniklerin ve uygulamaların olduğu, hangi uygulamalara hangi sırayla ihtiyaç duyulduğu belirlenir. Bunlar sistem geliştirme yaşam döngüsü ve yazılım geliştirme yaşam döngüsü içinde gerekecektir.

Hocanın verdiği tanımlar şöyledir:

- **Proje:** Planlanmış bir başlangıcı ve sonu olan, bir sonuca ulaşmak için gerçekleştirilen işlemler. Sistem geliştirme de aslında bir projedir.
- **Bilgi sistemi geliştirme projesi:** Bilgi sistemini üretmek üzere planlanmış alt işlemler.
- **Sistem geliştirme yaşam döngüsü (System Development Life Cycle, SDLC):** Donanım ve yazılımı birlikte kapsayan, bir bilgi sistemi projesinin geliştirilmesini anlatan genel terim. Bir temel yapı sağlar, kullanılacak yöntemleri, nasıl kontrol edileceğini ve nasıl bir kontrol listesi (checklist) oluşturulacağını verir.
- **Yazılım geliştirme yaşam döngüsü (Software Development Life Cycle):** Bu genel çerçeve içinde yalnızca yazılımın geliştirilmesi.

Yeni bir sistem geliştirmenin temel aktiviteleri analiz, tasarım ve gerçeklemedir (implementation). Gerçekleme aşaması kodlamayı, veri tabanının hazırlanmasını ve birim testlerin yapılmasını içerir. Analiz aşamasında şu sorular cevaplanır: Bilgi ihtiyaçları neler? Sistemde hangi işler gerçekleştiriliyor? Bu işlemlerle veri arasında nasıl bir ilişki var?

7. Temel ve Destek (Şemsiye) Aktiviteleri, Ölçme

Hoca, yazılım geliştirmenin temel işlemlerini ve destek aktivitelerini öğrencilere sorarak tekrar ettirdi (soruların kendisi Örnek Sorular bölümünde de verildi).

Temel işlemler sırasıyla gereksinim analizi, tasarım, gerçekleme ve testtir. Bunlar yazılım geliştirme yaşam döngüsünün temel işlemleridir.

Destek aktiviteleri, yani şemsiye aktiviteleri (umbrella activities), iyi bir analiz, tasarım ve gerçekleme yapmak için süreci iyileştiren işlemlerdir:

- **Proje yönetimi:** Büyük yazılım geliştirme işleri birer proje olarak düşünülür. Önce bir plan yapılır, sonra plan takip edilir, sonunda proje kapatılır. Bunların hepsi proje yönetimidir.
- **Risk yönetimi:** Proje yönetimi kapsamına da girer. Hangi riskler var, bunların takip edilmesi.
- **Kalite yönetimi:** Ürünün daha kaliteli olması için hangi gözden geçirmelerin yapılacağının planlanması.
- **Konfigürasyon yönetimi:** Değişiklik oldukça bunların takip edilmesi, elemanlar arasındaki ilişkilerin takip edilmesi.
- **Ölçme.**

7.1 Ölçmenin yeri

Hoca önceki derslerde mühendislik tanımını öne çıkardığını, bu tanımda ölçme ve tasarımın ön plana çıktığını hatırlattı. Tasarım zaten yazılım geliştirme döngüsünde bir temel madde olarak yer alıyor. Ölçme ise her aşamada yapılanlara uygulanır. Ölçmenin kullanım alanları:

- Algoritma karmaşıklığı ve program karmaşıklığı ölçüsü.
- Program büyüklüğü, kod büyüklüğü.
- Çalışanların veriminin ölçülmesi.
- Gerçekleştirilen işin büyüklüğünün ölçülmesi.
- Yeni bir işe başlarken eski ölçüm sonuçlarını kullanarak maliyet tahmini yapılması.

Planlama aşamasında yapılanlara proje yönetimi konusunda dönülecek. Hoca buradan, yazılım geliştirme sürecinde önem verilen başka bir konuya geçti: Kaliteyi sağlamak için yazılım evlerinin nasıl sertifikalandırıldığı.

8. CASE Araçları

Aradan sonra hoca, süreci iyileştirmek ve standartlaştırmak için kullanılan CASE araçlarına döndü. CASE, yazılım mühendisliği işlerinin bilgisayar yardımıyla gerçekleştirilmesidir. CASE ortamı hem temel adımları hem de destek adımlarını iyileştirir. Bunu yaparken grafik bir görüntü sağlayan bir ortam olmalıdır.

8.1 Üst, orta ve alt CASE

Hoca CASE araçlarını üç katmanda anlattı:

- **Üst CASE (upper CASE):** Diyagram çizimi ve tanımlama araçları.
- **Orta CASE:** Analiz ve tasarımı gerçekleştiren kısım. Hoca, öğrencilerden projede kullanmalarını istediği basit diyagram aracının bu tarafa düştüğünü belirtti.
- **Alt CASE (lower CASE):** Bir takım tanımlamalar yapıldıktan sonra kod üreten kod üreticiler ve kod çeviriciler.

Bu araçlar yazılım geliştirme sürecini düzenler ve projenin gerektirdiği standart tasarım tekniklerini ve yöntemlerini kullanır. Diyagram çizimi, tasarım ve kodlama işlemlerini yapabilmek için ansiklopedik bir arşivden (repository) beslenirler: veri modelleri, veri sözlüğü, işler, sistemler, fonksiyonlar.

> **Ek bilgi (derste anlatılmadı):** Literatürde en yaygın ayrım üst CASE ve alt CASE ikilisidir. Üst CASE gereksinim, analiz ve tasarım aşamalarını, alt CASE ise kodlama, test ve bakım aşamalarını destekler. “Orta CASE” adı bazı kaynaklarda analiz ve tasarımı destekleyen araçlar için kullanılır, ama büyük olasılıkla standart bir terim değildir. Bütün aşamaları ortak bir depo üzerinden birleştiren araçlara bütünleşik CASE (I-CASE, integrated CASE) denir.

8.2 Bütünleşik araçlar ve parça araçlar

Bütünleşik CASE araçları, bütün yazılım sistemini geliştirirken her aşamada destek olur, test sırasında kullanılan bölümleri de vardır. Hoca örnek olarak Rational Unified Process platformunu ve HP’nin bir yazılım geliştirme ürününü andı. Bunlar büyük projelerde, ancak kurumsal firmaların satın alabileceği güçte ürünlerdir.

Bunların yanında belirli evrelerde destek olan daha küçük, açık kaynaklı araçlar da vardır:

- Yalnızca analiz ve tasarım diyagramlarını çizmek için hazırlanmış olanlar (örneğin Violet).
- Yalnızca test için oluşturulmuş olanlar.
- Yalnızca tanımlanan algoritmaya göre kod üretenler.

8.3 Kod üreticiler ve dördüncü kuşak diller

Kod üreticilerde, yazılımı hızlandırmak için dördüncü kuşak dillerin (4GL) kullanımı söz konusu olabilir. Hoca, bu araçların yine bizim çizdiğimiz algoritma yoluyla gideceğini vurguladı. Bu yüzden dördüncü kuşak diller kullanıldığında da yapılan analiz ve tasarım evrelerine ihtiyaç vardır.

Hoca, yazılım geliştirirken belli bir standardı yakalamak için CASE araçlarının kullanılabildiğini, standartlardan kalite yönetimi konusunda bahsedeceğini söyledi.

8.4 Hangi araçları biliyorsunuz?

Hoca öğrencilere Violet dışında modellemeye destek olan bir ürün bilip bilmediklerini sordu. Öğrencilerden gelen iki örnek üzerinden şu ayrımı yaptı:

- **MS Project:** Yalnızca yazılım geliştirmeyle ilgili bir araç değildir. Planlama konusunda Gantt diyagramı ve PERT diyagramı çizmeye yardım eder, ama genel amaçlı, her türlü projeye destek sağlayan bir araçtır. Bu yüzden MS Project'e yazılım geliştirme süreci yardımcısı, yani CASE aracı denemez. Proje yönetim aracı denebilir.
- **SonarQube:** Hoca bunu test konusunda destek sağlayan bir ürün olarak yorumladı, yalnızca belli bir parçayı gerçekleştiren araçlara örnek verdi.

  Ders sonrası not: Altyazıda "Soner Küp" diye geçen ürün büyük olasılıkla SonarQube'dur. SonarQube bir test yürütme aracı değil, statik kod analizi ve kod kalitesi aracıdır. Test kapsamı (coverage) gibi ölçümleri başka araçlardan alıp raporlayabilir.

Hoca, yazılım mühendisliği dersinde böyle araçlarla işin kolaylaştırılabildiğini bilerek devam edilmesini istedi.

9. Süreç Sertifikasyonu: CMM ve CMMI

9.1 Neden süreç sertifikasyonu?

Hoca, yazılım tipleri ve paydaşların yanılgıları gibi konuları gördükten sonra, yazılım geliştiren ortamlarda nasıl bir standartlaşma ve iyileşme sağlandığına geçti. Yazılım geliştirme sürecinden bahsedilirken bu işi yapan yazılım evlerinin çalışmaları göz ardı edilemez. Gereksinim analizinde, tasarımda ve kodlamada standartlara uyarak yapılan iş iyileştirilebilir.

Hoca bunu, kalite dersinde de yaptığı gibi, üniversitelerle karşılaştırdı. Üniversitelerde bir bölümün ne kadar iyi olduğunu göstermek için MÜDEK gibi akreditasyonlar vardır. Yazılım geliştiren kurumların ne kadar iyi ve ne kadar standart olduğunu göstermek için de süreç sertifikasyonu vardır. Bu aslında yazılım kalite yönetimi konusudur, ama sürecin iyileştirilmesini anlamak için başlangıçta bilinmesinde fayda vardır.

Süreç sertifikasyonu farklı kurumlar tarafından verilir. Avrupa'da verilen bir sistem ve Amerika'da verilen bir sistem vardır. Amerika'daki sistem CMMI'dır.

> **Ders sonrası not:** Hoca açılımı "Capability Majority Model Integration" diye söyledi. Doğrusu "Capability Maturity Model Integration"dır (yetenek olgunluk modeli entegrasyonu). Altyazıda kısaltma "SMMI", "SMA" gibi biçimlerde de bozulmuştur. Avrupa'daki sistem derste adlandırılmadı. Büyük olasılıkla SPICE (ISO/IEC 15504, bugün ISO/IEC 33000 serisi) kastedilmiştir.

9.2 Sertifika ne işe yarar?

Bir yazılım firması süreçlerini yeterince uygulayıp uygulamadığını bağımsız kurumlara onaylatır ve bir sertifika alır. Bu sertifika bir takım ihalelere girmeyi kolaylaştırır. Örneğin askeri yazılımlar yapabilmek için CMMI 3. düzeyde olma şartı aranabilir. Askeri yazılımlar dışında da bazı ihalelerde sertifikalı yazılım firmalarıyla çalışma şartı olabilir.

Bu, çok küçük yazılım evlerinin mutlaka sertifika alma yüküne girmesi gerektiği anlamına gelmez. Böyle bir zorunluluk yoktur. Küçük firmalar kendilerine başka standartları hedef koyabilir. CMMI süreçlerini inceleyip kendilerine göre yorumlayarak da kullanabilirler.

Sistem ücretlidir. Sertifika almak isteyen yazılım evi başvurur ve kurallarını yerine getirmeye çalışır.

Hoca CMMI dışındaki standartları da andı: ISO 9001 standartları, yazılım geliştirme süreciyle ilgili standartlar ve proje yönetiminde PMI'ın geliştirdiği standartlar. Bunlar proje yönetimi ve kalite yönetimi konularında görülecek.

> **Ders sonrası not:** Hoca "Project Management Institution" dedi. Kurumun adı Project Management Institute'tur (PMI).

9.3 CMM'den CMMI'ya

Hocanın anlattığı tarihçe şöyledir:

- Başlangıçta yalnızca yazılım geliştirme süreci için CMM (Capability Maturity Model) vardı. Hoca, bunu bir döneme kadar CMM olarak anlattıklarını söyledi.
- Sonra yazılım evlerinde yürütülen her türlü aktivite birleştirildi ve bütünleşik CMMI kavramı ortaya çıktı. CMMI'da geliştirme (development), hizmet (service) ve ürün ve hizmet alımı (acquisition) için standartlar ortaya kondu.
- CMMI 2010'dan itibaren çevik süreçleri desteklemeye başladı.
- 2018'de bu üç alan tek bir olgunluk modelinde birleştirildi ve model sürekli iyileştirme hedeflerine göre yapılandırıldı.
- Ayrı yürütülen People CMM de son yıllarda CMMI'ya katıldı.

CMMI her düzeyde belirli süreç alanlarını kapsar. Her firma gerekli süreçleri kendi uygulamalarında sağlamayı hedefler.

People CMM'in eklenmesi şu demektir: Yazılım evi süreçleri uyguluyor, hizmet sunumunu ve yönetimini standartlara göre yürütüyor olabilir. Ama firmanın istenen düzeyde olabilmesi için çalışanlara sağladığı olanaklarda, kariyer planlarında ve çalışanların haklarında da düzeylere göre iyileştirme yapması gerekir. Hoca People CMM'de de katmanlar olduğunu anlattı. Örneğin ikinci düzeyde çalışanların işe alım süreçlerine ve doğru göreve yerleştirilmelerine bakılır. Daha üst düzeylerde kişilere kariyer planı yapılıp yapılmadığı, ölçümleme yapılıp yapılmadığı gibi değerlendirmeler eklenir.

> **Ders sonrası not:** Hocanın tarihçesi ana hatlarıyla doğrudur, ama tarihler altyazıdan doğrulanamaz. Büyük olasılıkla CMM 1.0 1991'de yayımlandı, CMMI 2000-2002 arasında çıktı, çevik uygulamalara yönelik açıklamalar 2010'da CMMI v1.3 ile eklendi ve CMMI V2.0 2018'de yayımlandı. Hocanın "97'ye kadar CMM olarak anlatıyorduk" ifadesi büyük olasılıkla CMMI'ya geçişten önceki dönemi kasteder. People CMM'in CMMI V2.0'a "People Management" gibi pratik alanlarla dahil edildiği de büyük olasılıktır.

10. CMMI Düzeyleri

Hoca, CMMI düzeylerinin sıfırıncı düzeyden beşinci düzeye kadar gittiğini söyledi ve her düzeyi slayt üzerinden anlattı.

Düzey	Ad	Hocanın anlatımı
0	Yürütülmeyen (Incomplete)	Hocaya göre hiç başvuru yapılmamış, henüz niyet de yok. Belki şirkette bir niyet vardır.
1	Giriş düzeyi (Initial / Performed)	Etkinlikler yürütülüyor: Projelerde analiz, tasarım, gerçekleme adımları kullanılıyor. Ama standartlaşma yok, her seferinde farklı hedeflerle yürüyor. Standartlaşma ve sertifika için bir çalışma başlatılmış olabilir.
2	Yönetilen / Yinelenebilir (Managed / Repeatable)	Temel planlama ve izleme yöntemleriyle önceki projelerdeki başarılar yeni projelerde tekrarlanabilir. Önceki başarı ve başarısızlık hikâyeleri dikkate alınır.
3	Tanımlanmış (Defined)	Kişi ve risk yönetimiyle proje yönetimi iyileştirilir. Tüm organizasyonda standart süreçler tanımlanmıştır.

Düzey	Ad	Hocanın anlatımı
4	Sayısal (nicel) yönetilen (Quantitatively Managed)	Süreçte ölçütler kullanılarak ölçme yapılır ve kalite yönetimi gerçekleştirilir. İlerleme sürekli izlenir, bütçe ve zaman hedeflerinden sapmalar erken belirlenir.
5	İyileştirilmiş (Optimizing)	Süreç yönetimi geçmiş deneyimlerin ışığında sürekli iyileştirilir.

Hoca "tekrarlanabilir (repeatable)" adının CMM aşamasındaki ad olduğunu belirtti.

10.1 Düzeyler üst üste eklenir

Düzeyler birbirinin üzerine eklenerek gider. Her düzeyde standartlaştırma ve iyileştirme bir öncekine katılır. Dördüncü düzeydeki bir firma ilk üç düzeyde yapılanları zaten yapıyordur.

- **İkinci düzey:** Temel işlemler vardır. Proje planlaması yapılır, analiz ve tasarım belgeleri geliştirilir. Konfigürasyon yönetimi mutlaka yapılır.
- **Üçüncü düzey:** Bunlara ek olarak risk yönetimi yapılır ve tüm organizasyonda standart süreçler tanımlıdır. İhalelerde genellikle üçüncü düzeyden itibaren olma isteği gelir.
- **Dördüncü düzey:** Hoca burada mühendislik tanımındaki ölçmeye döndü. Ölçme hem proje açısından (bütçe, zaman ve bunlardan sapmalar) hem de geliştirilen yazılım açısından yapılır. Yazılımla ilgili ilk akla gelen ölçü büyüklüktür. Sonra bu büyüklükteki bir projenin ne kadar sürede ve ne kadar iş gücüyle geliştirildiğine bakılır. Böylece maliyet tahmini, kişilerin kendi deneyimine dayanan rastgele tahminler yerine, önceki projelerin veri tabanına dayanan sayısal tahminlerle yapılır. Proje planlamasında da sayısal yönetime geçilmiş olur. Bu yüzden bu düzeye sayısal yönetilebilir düzey denir.
- **Beşinci düzey:** Yazılım geliştirme süreçleri, kullanılan teknikler ve araçlar için sürekli "nasıl daha iyi yaparız" sorusu sorulur. Çalışanlardan gelen yeni fikirlerin değerlendirilmesi de bu düzeyin özelliklerindendir.

  Hocanın vurgusu: İkinci düzeyde, tekrar edilebilir ve yönetilebilir olmak için konfigürasyon yönetimi şarttır. Üçüncü düzeyde risk yönetimi olmalıdır ve organizasyonun tamamında standart süreç adımları, süreç modeli takibi yapılmalıdır.

10.2 Anahtar süreç alanları

Bir yazılım evinin hangi düzeyde olduğunu belirlemek için anahtar süreç alanlarına (key process areas, KPA) bakılır. Hoca her düzeyde nelere bakıldığını şöyle örnekledi:

- **"Birinci düzeydeyim" diyen firma:** Mevcut durumda her projede yazılım geliştirme süreç adımlarını yürütüyor mu? Sonrası için hazırlık planları var mı?
- **"İkinci düzeydeyim" diyen firma:** Bütün süreç adımlarını işlediğinin belgesi gerekir. Önceki projelerden geri besleme alınmaya başlanmış mı? Geliştirilen bütün yazılımların hangi aşamalardan geçtiğini ve hangi yarı ürünleri ürettiğini izleyen bir değişiklik yönetimi, yani konfigürasyon yönetimi sistemi var mı?
- **Üçüncü düzey:** Risk yönetimi var mı ve organizasyonun hepsinde standart süreç modeli izleniyor mu?

  Ders sonrası not: Hoca 0. düzeyi "hiç başvuru yapılmamış" diye anlattı. Bu doğru değil. CMMI'da 0. düzey (Incomplete), bir sürecin hiç ya da yalnızca kısmen yürütüldüğü durumdur ve bir sertifika başvurusuyla ilgisi yoktur. Büyük olasılıkla 0 düzeyi yalnızca süreç alanı bazındaki yetenek (capability) düzeylerinde vardır. Kurumsal olgunluk (maturity) düzeyleri 1'den başlar. 1. düzey için slayttaki "iş şansa ve anahtar kişilere kalmış" tanımı daha doğrudur: Süreçler gelişigüzel (ad hoc) yürür, başarı kişilere bağlıdır.

Ek bilgi (derste anlatılmadı): CMMI iki tür düzey tanımlar. Yetenek (capability) düzeyleri tek tek süreç alanlarının ne kadar iyi uygulandığını gösterir ve büyük olasılıkla 0-3 arasındadır. Olgunluk (maturity) düzeyleri ise kurumun bütününü 1-5 arasında derecelendirir. Derste anlatılan 1-5 tablosu olgunluk düzeyleridir. "Anahtar süreç alanı (KPA)" terimi asıl olarak CMM'den gelir, CMMI'da bunun yerine büyük olasılıkla "süreç alanı (process area)" ve V2.0'da "pratik alanı (practice area)" kullanılır.

10.3 Süreç modelleri ile bağlantı

Hoca bu noktada geçen hafta görülen süreç modellerini hatırlattı: klasik modeller, evrimsel modeller, artımlı modeller ve çevik modeller. Çevik modellerin birçoğunun zaten artımlı model olduğunu belirtti. Üçüncü düzeyde organizasyon genelinde izlenmesi beklenen "standart süreç modeli" bu modellerden biridir.

11. Türkiye'deki Durum

Hoca, Türkiye'de önemli olanın 3. düzey ve üstündeki kurumlar olduğunu söyleyerek bir liste gösterdi. Aşağıdaki bilgiler hocanın derste aktardıklarıdır. Sertifikalar süreli olduğu için güncel durum farklı olabilir.

- **Milsoft:** Ankara'da askeri yazılım (military software) yapan bir firma. Hocaya göre sürekli 5. düzeyde.
- **TÜBİTAK BİLGEM:** 5. düzeydeydi.
- **ASELSAN, Cybersoft, HAVELSAN, Koç Sistem:** 3. düzeydeydi.
  - Koç Sistem'de uzun yıllar kalite grubu çalışmalar yapmıştır.
  - Cybersoft bankacılık yazılımı yapan bir firmadır. Hocaya göre şirket sahipleri de yazılım mühendisidir ve süreçlere önem vererek çalışırlar.
  - HAVELSAN askeri bir firma olduğu için yazılım ürünlerini de 3. düzeyde yürütür.
- **ETİYA:** 2020'de 3. düzeyi almıştı. Teknoparklarda yazılım geliştiren bazı firmalar da çalışmalarını 3. düzeye göre yürütüyor olabilir.
- **Siemens:** Listede olmayan bir şirket. CMMI kullanıyor. Ayrıca test işlemlerinde yeterli olgunluğa ve iyileştirmeye ulaşılıp ulaşılmadığını gösteren TMMi (Test Maturity Model integration) sertifikasını almış. Hocaya göre çevik yaklaşımlarla birlikte TMMi 5'i uygulama aşamasındalar.

  Ders sonrası not: Altyazıda "COÇ SİSTEM" Koç Sistem, "Etia" ise ETİYA olarak düzeltildi. TMMi, CMMI'ya benzer biçimde test sürecinin olgunluğunu büyük olasılıkla 5 düzeyde derecelendiren ayrı bir modeldir.

12. Nesneye Yönelik Gereksinim Analizine Giriş: Kullanıcı Hikâyeleri ve Use Case

12.1 Kaynak ve amaç

Bir sonraki hafta gereksinim analizine Pressman'ın kitabından, adımları göstererek devam edilecekti. Hoca, kuramsal adımlara geçmeden önce nesneye yönelik analize küçük bir giriş yapmak istedi. Bunun için Craig Larman'ın hazırladığı sunumları kullandı. Larman'ın kitaplarının ve web üzerindeki sunumlarının temel nesneye yönelik analiz ve tasarım konusunda faydalı olacağını daha önce de söylemişti.

12.2 Kullanıcı hikâyeleri

Fonksiyonel gereksinimleri yakalamanın bir yolu kullanıcı hikâyelerini (user stories) dinlemektir. Bir yazılım geliştirilecekse, yazılımın geliştirileceği sistem hakkında bilgi alınmalı ve farklı seviyelerde ayrıntı istenmelidir.

Hoca burada iki noktayı birbirine bağladı. Çevik yaklaşımda "basitten başlayın" deniyordu. Bu derse de kullanıcının çok karmaşık hikâyeler anlatmasının tehlikeli olduğunu gösteren salıncak hikâyesiyle başlanmıştı. Doğru yol şudur: Önce temel olarak ne istendiği alınır. Sonra

bunun use case'i ve temel bir diyagramı oluşturulur. Ardından kademe kademe ayrıntıya girilir. Bu yüzden farklı use case seviyeleri tanımlanır ve kullanıcı hikâyeleri use case'lere temel bir formatta yazılır.

Use case diyagramlarıyla ilgili iki beceri beklenir:

- **Okuma:** Size verilmiş bir use case diyagramının ne dediğini anlayabilmek.
- **Yazma:** Bir müşteri ya da iş analisti bir şey anlatırken, dinlerken use case diyagramını yazabilmek.

  Hocanın vurgusu: Sınavda verilen bir senaryonun use case diyagramını ve use case belgesini yazabilecek durumda olmalısınız.

12.3 Use case nedir?

Use case, sistemi kullananların gerçek hayatta bir hedefi gerçekleştirmek için ne yaptıklarını anlatır. Hoca, Larman'ın klasik örneği olan video kiralama şirketini kullandı ve derste ileride başka bir uygulama üzerinde de aynı şeyi göstereceklerini söyledi.

Bir hikâye anlatıldığında şunlar çıkarılır:

- **Aktörler:** Birincil aktörler (primary actors) kimler, yani işi gerçekleştiren kim? Dış elemanlar kimler? Destek aktörleri kimler?
- **Olayın kısa tanımı:** Müşteriden dinlerken bu işin ne olduğu.
- **Sistemin girdisi, işlemi ve çıktısı:** Yazılım sistemi nasıl çalışacak, bilgi olarak ne alacak, ne işleyecek, ne verecek.
- **Sıralı iş adımları:** Formal olmayan bir senaryoda da belirli bir sıra vardır. Use case'i tamamlamak için anlatım olarak verilen paragraftan sırayla gerçekleşen iş adımları çıkarılır.

Video kiralama hikâyesinin kısa tanımı şöyledir:

1. Müşteri gelir, video kiralamak ister.
2. Görevli müşterinin kimliğini (ID) ve her videonun ID'sini girer.
3. Sistem kiralama bilgisini çıktı olarak verir.
4. Sistemden bir kiralama raporu istenebilir: Kimler kiralamış, hangi müşteriler hangi videoları almış.

Use case'ler yalnızca nesneye yönelik yaklaşımda değil, her yaklaşımda kullanılabilir. Nesneye yönelik yaklaşımda ise kullanıcı hikâyelerinden sonra bir use case diyagramı çizmek, sınıf diyagramlarına ve hangi aktörlerin hangi işleri sırayla yaptığını gösteren sıralama diyagramlarına geçmeden önce hedefe daha çabuk ulaşmayı sağlar.

12.4 Gereksinim analizinde use case'in yeri

Gereksinim analizinde sıra şöyledir:

1. Bilgi toplanır.
2. Toplanan bilgi modellenir, yani use case diyagramı çizilir.
3. Diyagram farklı bakış açılarına sunulur. Farklı bakış açısı, müşteri tarafının farklı kesimleri demektir. Örneğin bir satış-sipariş işleminde stokta çalışan, fatura kesen ve müşteri temsilcisi işe farklı açılardan bilgi verir.
4. Bu bilgilerin birbiriyle tutarlı olması sağlanır. Bunun için bütün paydaşların fikri alınır.

12.5 Use case diyagramında ayrıntı düzeyi

Hoca "bütün use case'ler faydalı mıdır?" sorusunu sordu ve şöyle cevapladı: Karmaşık bir problemde ilk başta bütün hedefleri birden alan çok karmaşık bir use case çizilmemelidir. Önce bütün organizasyonu kapsayan, temel işlemleri gösteren bir diyagram oluşturulur. Sonra bu diyagram alt diyagramlarla ayrıntılandırılır. Bu tek adımda olmaz, basitten ayrıntıya doğru adım adım gidilir.

Hoca bunu veri akış diyagramlarındaki (data flow diagram) seviyelendirmeye benzetti: Önce bir özet diyagram oluşturulur, sonra ondan daha ayrıntılı diyagramlar çıkarılır. Slayttaki video

kiralama diyagramı böyle bir taslak yapıdaydı. Ardından use case'lerin ve aktörlerin neler olduğu belirlenir, ayrıntılı analiz yapıldıkça alt seviyelere geçilir.

13. Fonksiyonel ve Fonksiyonel Olmayan Gereksinimler

Hoca, bir sonraki hafta daha ayrıntılı görülecek iki kavramı tanıttı.

Fonksiyonel gereksinimler (functional requirements): Yazılım hazırlanması istenen sistemde neler yapıldığının, hangi işlemlerin yapıldığının belirtilmesidir. Yapılan bu temel işlemlere fonksiyonel gereksinim denir. Fonksiyonel gereksinimler işin kademe kademe nasıl yapıldığını anlatır.

Fonksiyonel olmayan gereksinimler (non-functional requirements): Daha genel yaklaşımları, daha teknik kuralları ve standartlara uygun hareket etmeyi sağlayan kurallardır. Kaliteyle ilgilidirler. Örnekler:

- Performansın iyi olması.
- Geri dönüş (yanıt) hızının iyi olması.
- Güvenliğin belirli bir şekilde sağlanması.

Gereksinim analizinde hem fonksiyonel hem de fonksiyonel olmayan gereksinimler ayrıntılı bir analiz sonucunda ortaya çıkarılmalıdır.

14. Use Case Belgesinin Yapısı

Hoca son olarak slayttaki use case belgesinin genel yapısını gösterdi. Video kiralama örneğinde belge şöyledir:

```
Use Case UC1: Rent Video (Video Kiralama)
Seviye (Level): Kullanıcı hedefi seviyesi (User-level goal)
Birincil aktör (Primary Actor): Görevli (Clerk)
Ön koşullar (Preconditions):
  - Görevli kendini tanıtmış ve sisteme giriş yapmış olmalı
Paydaşlar ve ilgileri (Stakeholders and Interests):
  - Görevli: Doğru ve hızlı giriş yapmak ister.
  - Müşteri: Videoların hızlı verilmesini, çok uğraştırılmamayı ister.
  - Muhasebe: Kayıtların doğru girilmesini ister.
  - Pazarlama: (müşteri alışkanlıklarıyla ilgili bir ilgi)
Ana başarı senaryosu: Sırayla gerçekleştirilecek işlemler, algoritma düzeyinde
```

Hoca, basit bir firmada muhasebe ve pazarlama kısımlarının zaten olmayacağını, her şeyi görevlinin kendisinin takip edeceğini ekledi.

Use case belgesinde kısaca şunlar istenir: use case'in adı, seviyesi, birincil aktör, use case'in başlaması için ön koşullar, diğer paydaşlar ve sistemden ne istedikleri, ve sırayla gerçekleştirilecek işlemlerin algoritma düzeyinde verilmesi.

Use case belgesi ile diyagram paralel yürür. Önce diyagramı mı, belgeyi mi yazacağınıza siz karar verirsiniz. Ama ilk başta basit, taslak bir diyagramla başlanır. Sonra ayrıntılı diyagram çizilir ya da belgede iyileştirmeler yapılır. Bu, use case hikâyelerini anlatmanın bir yoludur ve gereksinimleri iyi belirlemek için deneme yapmak gerekir.

> **Ek bilgi (derste anlatılmadı):** Larman bu tür ayrıntılı belgeye "fully dressed" use case der. Belgede derste geçenlerin yanında büyük olasılıkla son koşullar (başarı garantisi), alternatif akışlar (extensions), özel gereksinimler ve teknoloji listesi gibi bölümler de bulunur. Slaytta seviye için geçen "EBP", büyük olasılıkla Elementary Business Process (temel iş süreci) kısaltmasıdır: Bir kişinin tek oturumda, tek yerde yaptığı ve ölçülebilir bir iş değeri üreten işlem.

Örnek Sorular

1. (derste sözlü, 01:03:14) Sistem geliştirme sırasında yazılım sisteminin geliştirilmesiyle ilgili temel işlemler nelerdir?

Derste verilen cevap: Sırasıyla gereksinim analizi, tasarım, gerçekleme (kodlama) ve test. Hoca bunların yazılım geliştirme yaşam döngüsünün temel işlemleri olduğunu onayladı.

2. (derste sözlü, 01:04:30) Temel işlemlere destek sağlayan şemsiye aktiviteleri (umbrella activities) nelerdir?

Derste verilen cevap: Proje yönetimi (planlama), kalite yönetimi, risk yönetimi ve yazılım konfigürasyon yönetimi. Hoca bunlara ölçmeyi ekledi ve her birini açıkladı: Proje yönetimi planın yapılması, takibi ve kapatılması; risk yönetimi risklerin takibi; kalite yönetimi gözden geçirmelerin planlanması; konfigürasyon yönetimi değişikliklerin ve elemanlar arası ilişkilerin takibi.

3. (derste sözlü, 01:43:40) Nesneye yönelik modellemede Violet dışında destek olan bir araç biliyor musunuz?

Derste verilen cevap: Öğrencilerden MS Project ve SonarQube örnekleri geldi. Hoca MS Project'in bir CASE aracı değil, Gantt ve PERT diyagramları çizen genel amaçlı bir proje yönetim aracı olduğunu söyledi. SonarQube'u ise yalnızca belirli bir aşamayı destekleyen parça araçlara örnek verdi (bkz. Bölüm 8.4).

4. (derste sözlü, 02:15:01) Bütün use case'ler faydalı mıdır? Karmaşık bir problemde use case diyagramına nasıl başlanır?

Derste verilen cevap: Karmaşık bir problemde ilk başta bütün hedefleri kapsayan tek bir karmaşık use case çizilmemelidir. Önce bütün organizasyonun temel işlemlerini gösteren özet bir diyagram çizilir, sonra veri akış diyagramlarındaki seviyelendirmeye benzer biçimde alt diyagramlarla adım adım ayrıntıya inilir.

Dersin Toparlaması

Ders iki parçadan oluştu. İlk parçada dönem projesinin teknik çerçevesi çizildi: Proje baştan sona nesneye yönelik yapılacak. Proje planı, use case belgesi ve diyagramı, sınıf, sıralama, etkinlik ve durum diyagramları hazırlanacak. Kullanıldıysa tasarım kalıbı belirtilecek, en az bir modül kodlanıp iki birim testiyle sınanacak. Diyagramlar standartlık ve diyagramlar arası tutarlılık için bir CASE aracıyla çizilecek. Üç konu (seyahat acentesi, yazılım evi insan kaynakları, medya takip) yalnızca ipuçlarıyla verildi. Modülleri belirlemek ve projeyi küçültmeyen varsayımlar yazmak gruplara bırakıldı.

İkinci parçada salıncak hikâyesiyle analiz ve tasarımın önemi gösterildi, ardından sistem geliştirme yaşam döngüsü, temel ve şemsiye aktiviteleri ve ölçme tekrar edildi. CASE araçları üst, orta ve alt katmanlarıyla, bütünleşik ve parça araçlar ayrımıyla anlatıldı. Süreç kalitesinin kurum düzeyindeki karşılığı olarak CMMI ve düzeyleri işlendi: 2. düzeyde konfigürasyon yönetimi, 3. düzeyde risk yönetimi ve organizasyon çapında standart süreç, 4. düzeyde ölçmeye dayalı yönetim, 5. düzeyde sürekli iyileştirme. Son olarak Larman'ın video kiralama örneğiyle kullanıcı hikâyelerinden use case'e geçiş, fonksiyonel ve fonksiyonel olmayan gereksinimler ve use case belgesinin yapısı tanıtıldı. Bir sonraki hafta gereksinim mühendisliğine bilgi toplama aşamasından başlanacak ve Pressman ile Sommerville'in kitaplarındaki ilgili bölümler işlenecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.